

Experiment 2

Arman Imam

EEE3342C

25Fall 0011

9/19/2025

## Equipment:

Computer with Xilinx ISE software, Vivado 2017.4 specifically.

## Objective

The objective of Experiment 2 is to introduce the user to Testbenches and how to apply multiple logic gates together to obtain an output via inputs. A simple Boolean network test bench, and complex Boolean network test bench were built and tested as schematics in the experiment. Circuit debugging for an example test bench was also tested.

## Design Steps:

I began by creating a folder that held 3 separate projects: one for the simple schematic, complex schematic, and circuit debug. For the simple schematic projects, I've written down the code that reflects the AND, and 2 NOR gate order for the source code. For the complex schematic, I've reflected the NAND, AND, NOT, OR, and XOR gate, in the source code. In the test bench, I've coded a loop that reflects the number inputs and outputs, to account for every iteration of 1s and 0s. For circuit debugging, I've analyzed the X and Y outputs to discern a pattern, which has allowed me to find the error in the code given.

## Part 1:

N/A

## Part 2: Simple Boolean Network Testbench

- RTL code

```
//Lab 2 Part 2
module Lab2_source(
    input a, b, c, d,
    output y
);
    wire top,bottom;

    assign top = a & b;
    assign bottom = ~(c | d);
    assign y = ~(top | bottom);

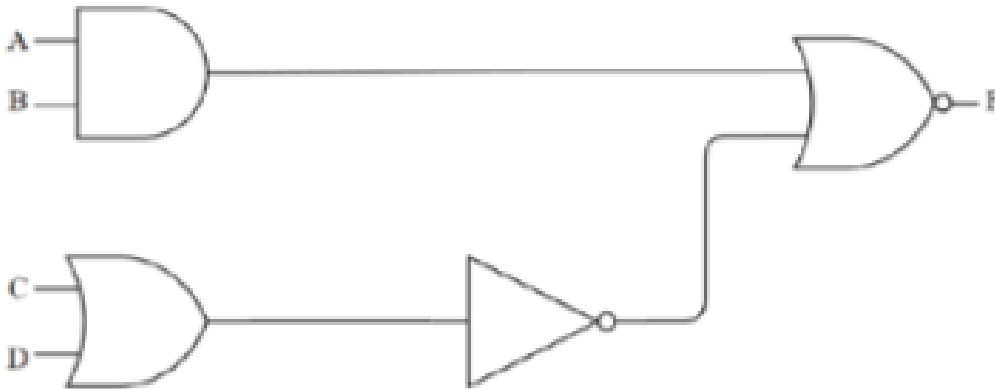
endmodule
```

- Test bench code

```
22 //Lab 2 Part 2
23 module Lab2_sim();
24
25     parameter NUM_INP =4;
26     reg a, b, c, d;
27     reg[NUM_INP-1:0] cnt;
28     wire y;
29     integer i;
30
31     Lab2_source UUT(.a(a), .b(b), .c(c), .d(d), .y(y));
32     initial begin
33         cnt = 0;
34         for(i = 0; i < 2**NUM_INP; i = i + 1)begin
35             a = cnt[1];
36             b = cnt[0];
37             c = cnt[1];
38             d = cnt[0];
39             cnt = cnt + 1;
40             #10;
41         end
42     end
43 endmodule
44
```

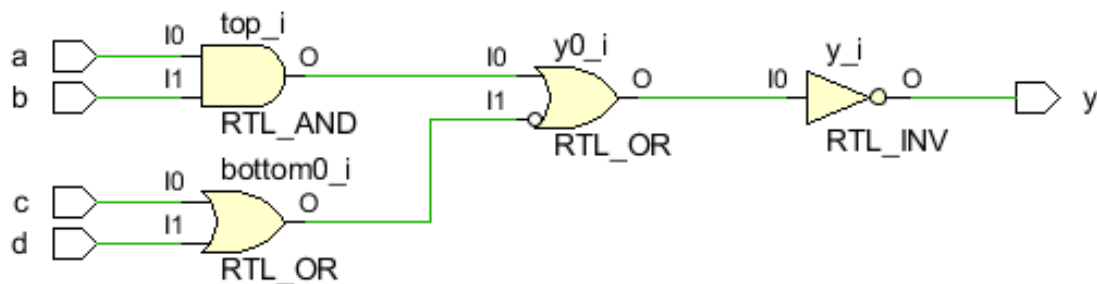
- Logic Diagram:

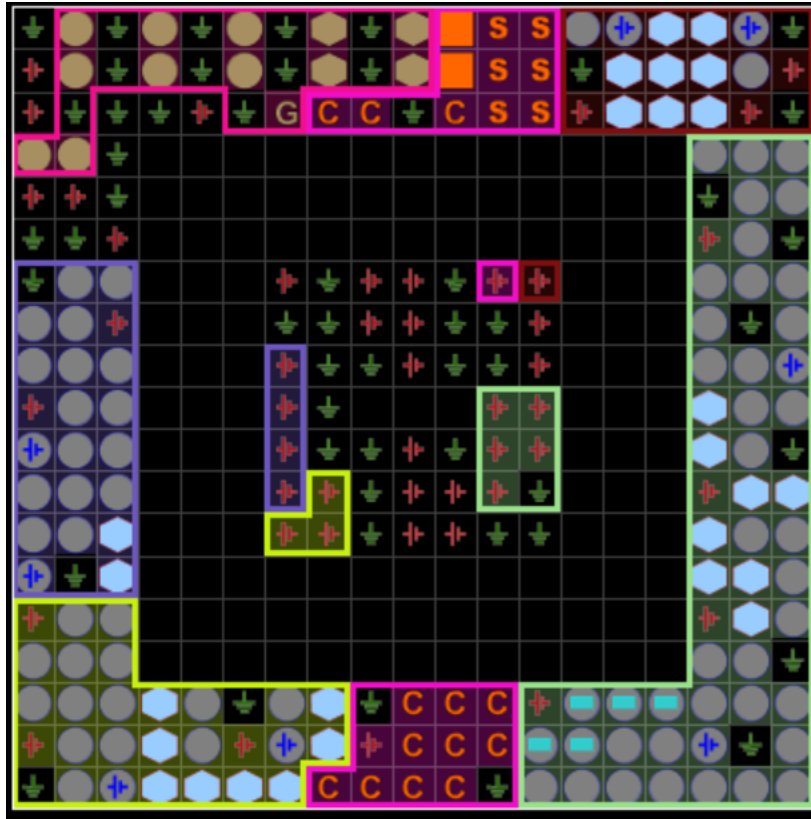
This diagram was given from experiment 2 brief for replication



- Design Specification

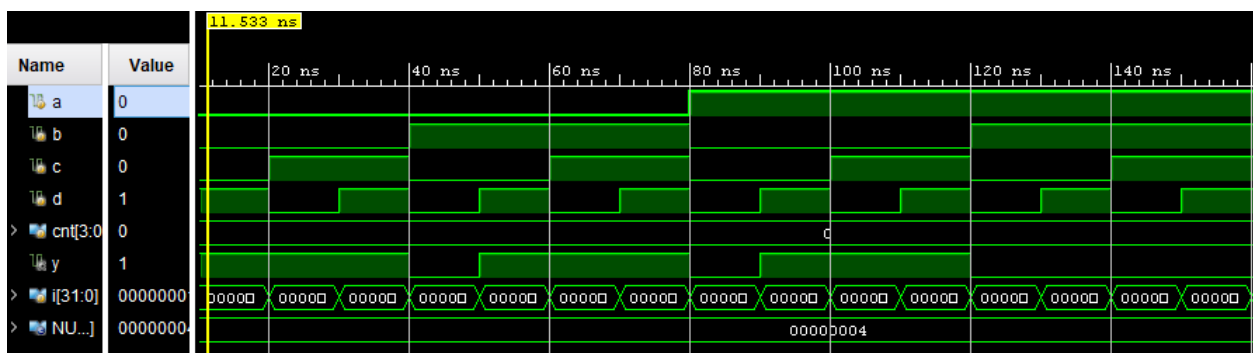
The experiment calls for 4 inputs(a,b,c,d), along with an AND, and 2 NOR gates, and 1 output(y).





## • Results Statement:

The result give 16 different combinations, where y is shown to be 1, except for when the combination is (0,0,0,0), (0,1,0,0), (1,0,0,0), and (1,1,0,0) up to (1,1,1,1). Form is in (a,b,c,d,e)



|   |   |   |   |   |
|---|---|---|---|---|
| a | b | c | d | y |
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 1 |

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 1 |
| 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 1 | 1 |
| 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 | 0 |

## Part 3: Complex Boolean Network Testbench

### • RTL code

```

22 //Lab 2 Part 3
23 module Lab2_source(
24     input a, b, c, d, e,
25     output f
26 );
27     wire or_out, and_out;
28     assign or_out = ~(a & b) | c;
29     assign and_out = ~c & ~(d | e);
30
31     assign f = or_out ^ and_out;
32
33 endmodule

```

### • Test bench code

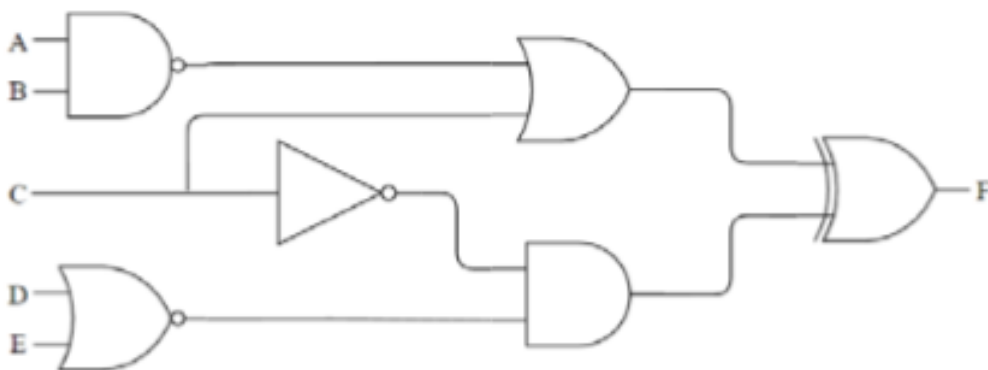
```

22 //Lab 2 Part 3
23 module Lab2_sim();
24
25 parameter NUM_INP =5;
26 reg a, b, c, d, e;
27 reg[NUM_INP-1:0] cnt;
28 wire f;
29 integer i;
30
31 Lab2_source UUT(.a(a), .b(b), .c(c), .d(d), .e(e), .f(f));
32 initial begin
33     cnt = 0;
34     for(i = 0; i < 2**NUM_INP; i = i + 1)begin
35         {a,b,c,d,e} = i;
36         #10;
37     end
38 end
39 endmodule
40

```

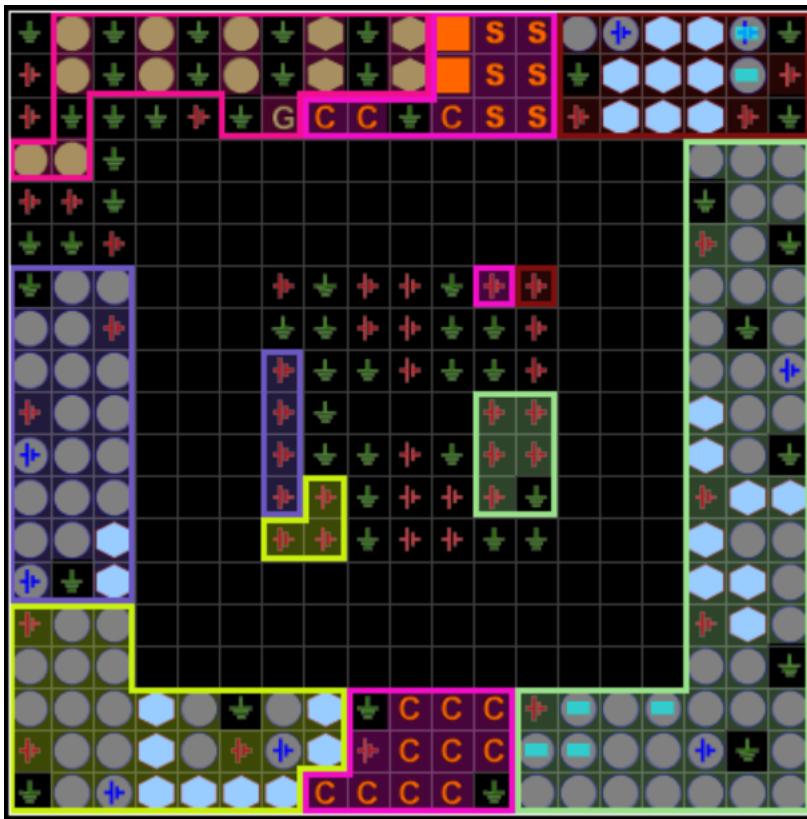
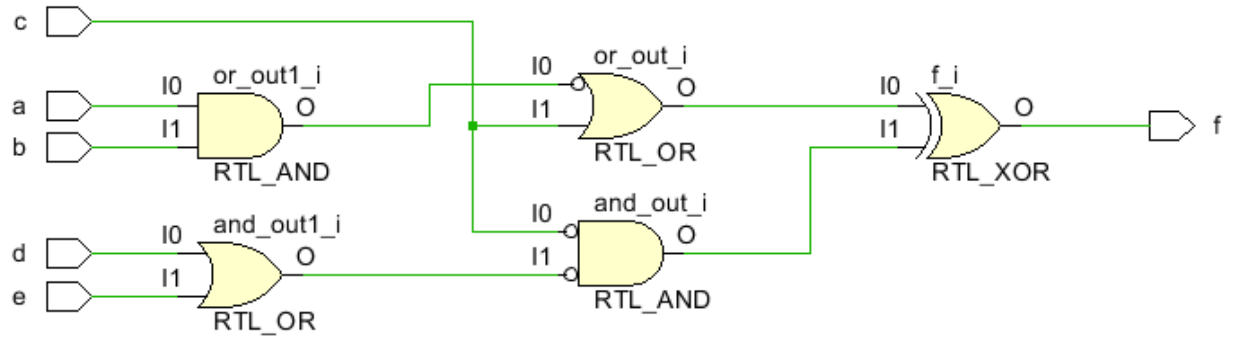
## •Logic Diagram

This diagram was given from experiment 2 brief for replication.



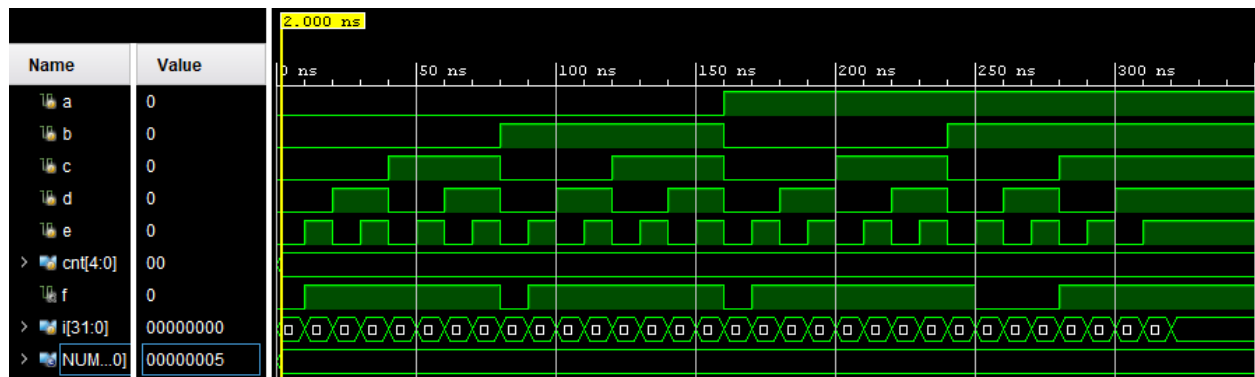
## •Design Specification Plan

The experiment calls for 5 inputs(a,b,c,d,e), along with a NAND, NOR, NOT, AND, OR, and XOR gate, with 1 output(f).



### • Results Statement:

The result gives 32 different combinations, where y is shown to be 1, except for when the combination is (0,0,0,0,0), (0,1,0,0,0), (1,0,0,0,0), (1,1,0,0,1), (1,1,0,1,0), and (1,1,0,1,1). Form is in (a,b,c,d,e)



| a | b | c | d | e | y |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 0 | 1 | 0 | 1 |
| 0 | 0 | 0 | 1 | 1 | 1 |
| 0 | 0 | 1 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 | 1 | 1 |
| 0 | 0 | 1 | 1 | 0 | 1 |
| 0 | 0 | 1 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 1 | 1 | 1 |
| 0 | 1 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 1 | 0 | 1 |
| 0 | 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 1 |
| 1 | 0 | 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 1 | 1 |
| 1 | 0 | 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 | 1 | 1 |
| 1 | 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 1 | 1 | 1 | 1 |
| 1 | 1 | 0 | 0 | 0 | 1 |
| 1 | 1 | 0 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 1 | 0 | 1 |

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 |
|---|---|---|---|---|---|

## Part 4: Circuit Debug

- Updated RTL code

```

22 //Lab 2 Part 4
23 module logic_top(
24     input A, B, C,
25     output X, Y
26 );
27     wire w1, w2, w3;
28     assign w1 = A ^ B;
29     assign w2 = C & w1;
30     assign w3 = A & B;
31     assign X = C ^ w1;
32     assign Y = w2 | w3;
33 endmodule
34
35

```

- Bug found:

For assign X, it should've been  $C \wedge w1$  instead of  $C | w1$ , and for assign Y, it should've been  $w2 | w3$  instead of  $w2 \& w3$ .

- Test bench code

```

22 //Lab 2 Part 4
23 module Lab2_sim();
24
25     parameter NUM_INP =3;
26     reg A,B,C;
27     reg[NUM_INP-1:0] cnt;
28     wire X,Y;
29     integer i;
30
31     logic_top UUT(.A(A), .B(B), .C(C), .X(X), .Y(Y));
32     initial begin
33         cnt = 0;
34         for(i = 0; i < 2**NUM_INP; i = i + 1)begin
35             {A,B,C} = i;
36             #10;
37         end
38     end
39 endmodule
40

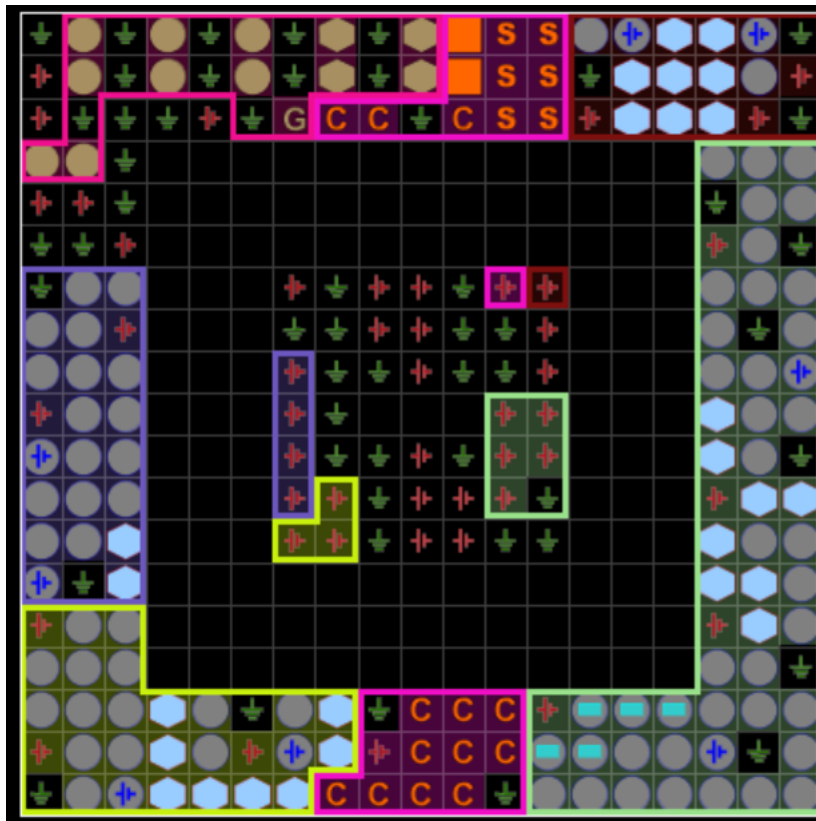
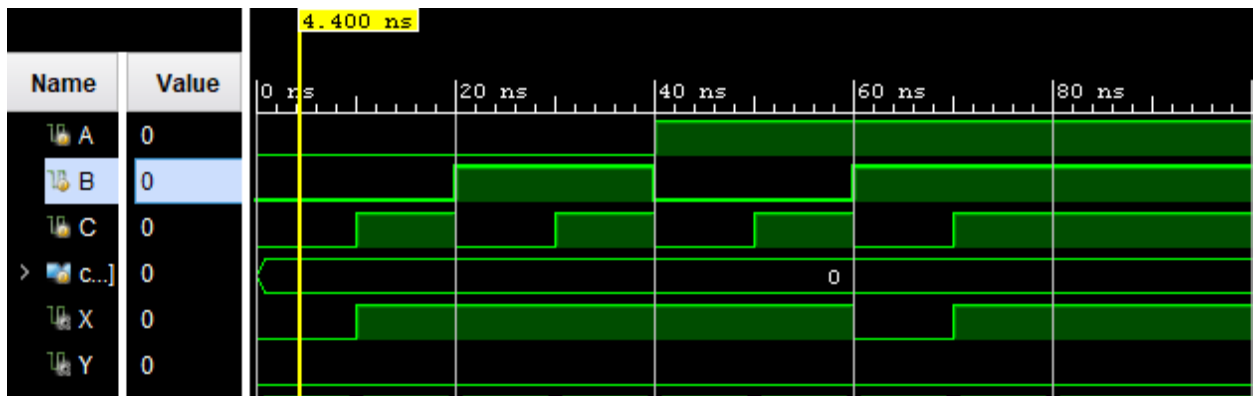
```

Truth Table:

| Inputs   |          |          | Outputs  |          |
|----------|----------|----------|----------|----------|
| <u>A</u> | <u>B</u> | <u>C</u> | <u>X</u> | <u>Y</u> |
| 0        | 0        | 0        | 0        | 0        |
| 0        | 0        | 1        | 1        | 0        |
| 0        | 1        | 0        | 1        | 0        |
| 0        | 1        | 1        | 0        | 1        |
| 1        | 0        | 0        | 1        | 0        |
| 1        | 0        | 1        | 0        | 1        |
| 1        | 1        | 0        | 0        | 1        |
| 1        | 1        | 1        | 1        | 1        |

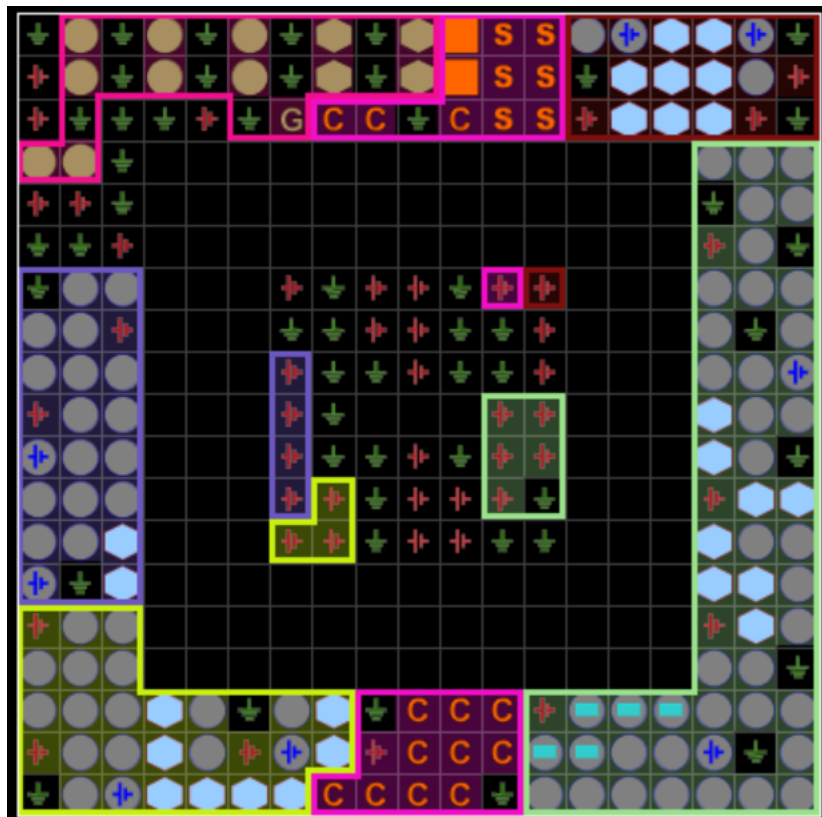
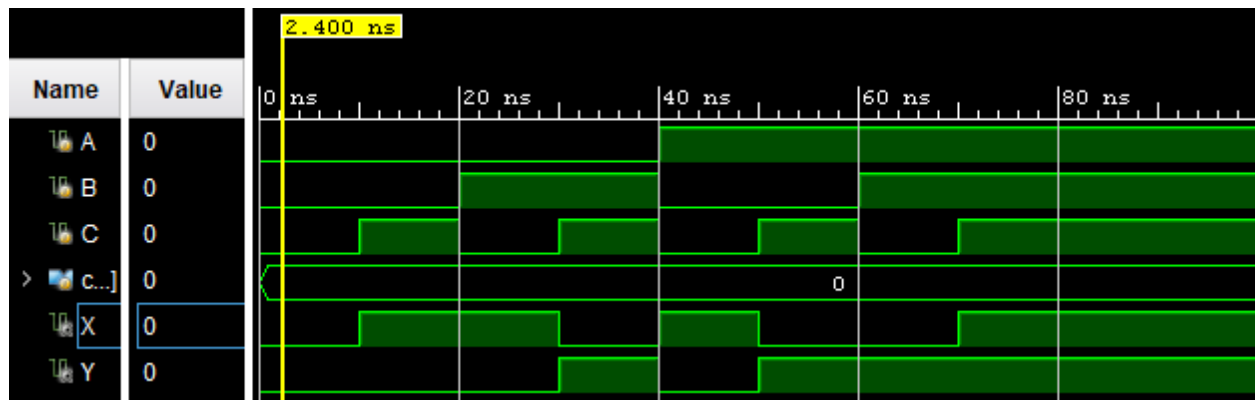
- Vivado waveform before debugging

The results for X and Y don't match the output of the truth table. Here, Y is always 0 no matter the input, and X is only 0 in two combinations.

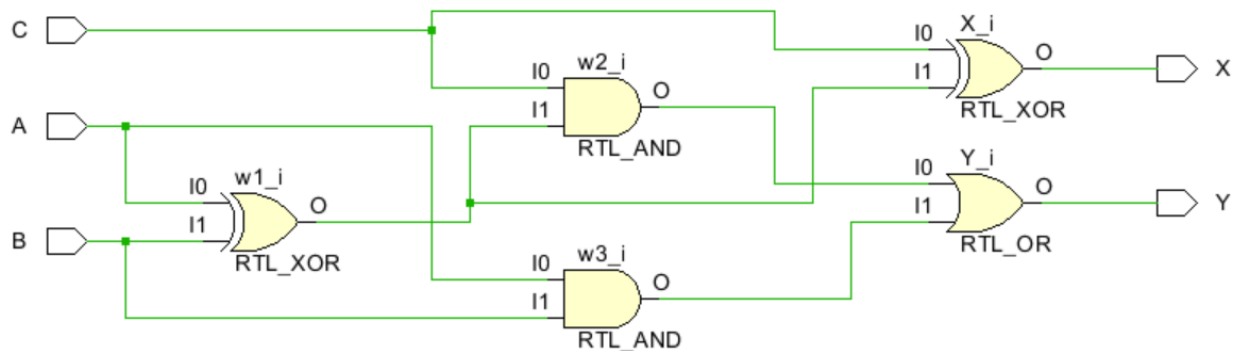
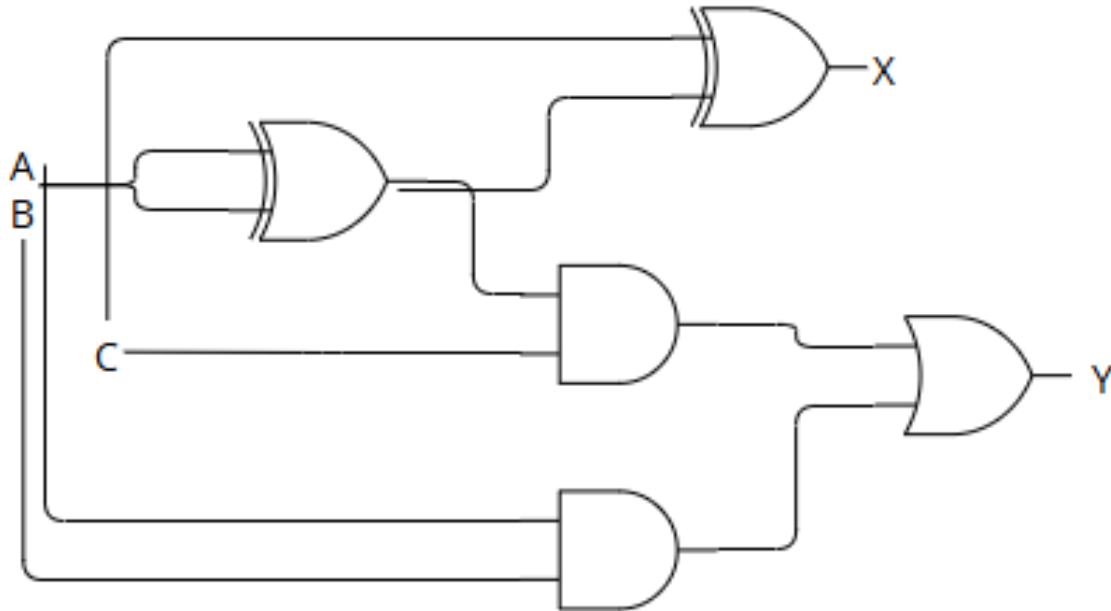


- Vivado waveform after debugging

After assigning  $X = C \wedge w1$ , and  $Y = w2 \mid w3$ , the output now reflects the output of the truth table.



- Logic diagram of updated RTL code



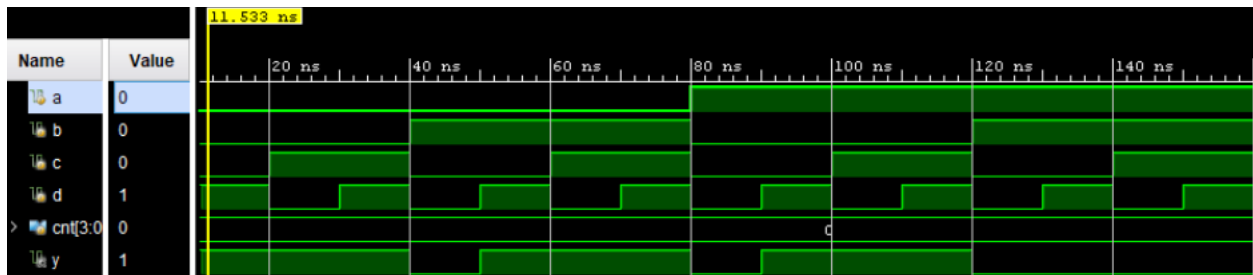
### Overall Lab Q&A:

1. The type of test bench I would write if I wanted to test all possible permutations would be an exhaustive test bench. This can involve a for loop, that goes through each binary combination based on the number of inputs;  $2^{(\text{number of inputs})}$  number of combinations.

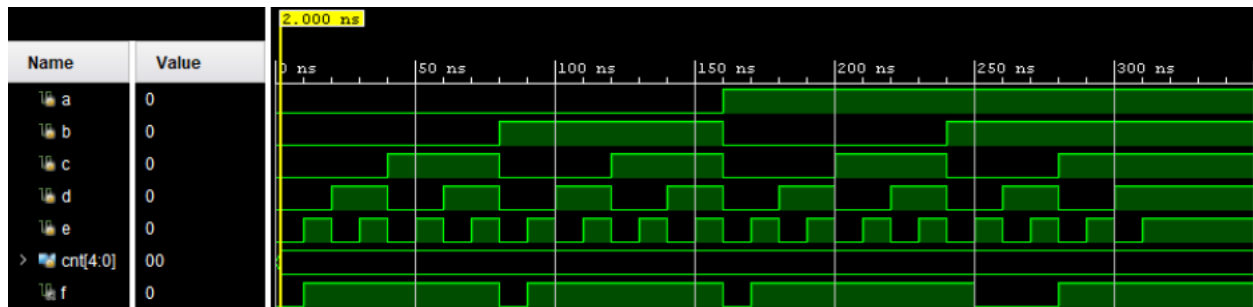
2. The type of test bench I would write if I wanted to test a specific input would be a direct test bench. I would put (0,0),(1,0),(0,1) and (1,1) binary inputs manually, and go through each iteration every 10 nanoseconds.
3. The Verilog data type for inputs is wire and outputs are registers.
4. The purpose of a delay in a test bench is to allow the use to have a clear view of how the code changes through each iteration or ensure correct verification of the design's behavior over time.
5. The test bench is not a piece of hardware generated by Vivado, because the test bench was only used and accessed in software.

## Test Plan:

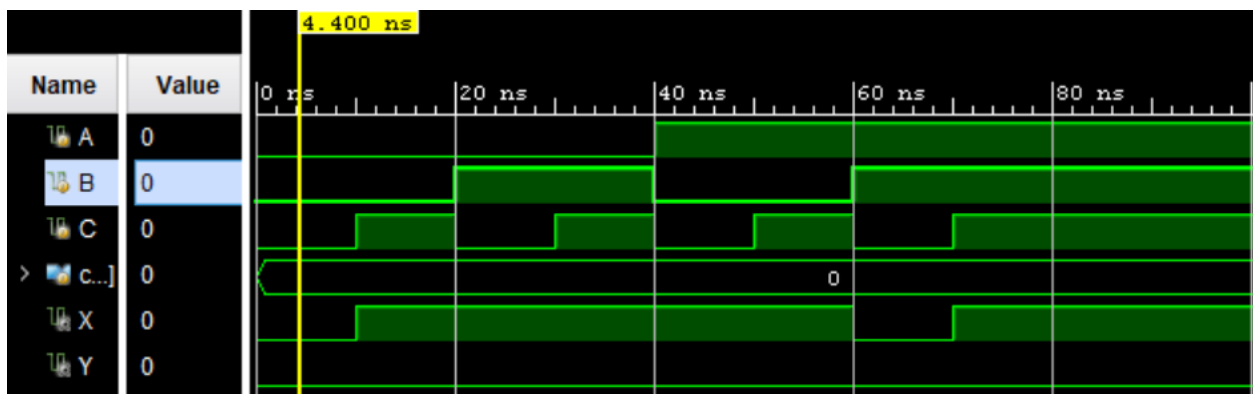
First, I checked if the schematics align with what's suppose to be showing, then the design was tested by running a simulation based off the test bench, which goes through every possible input combinations. After the tests, I verified that the truth tables match the design's waveforms.



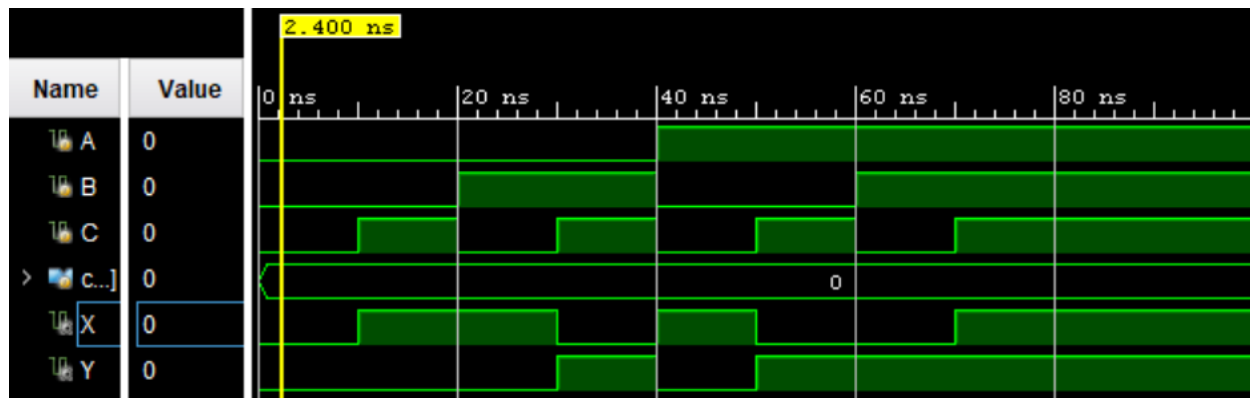
Test Bench for 4-input, for AND and 2 NOR gates



Test Bench for 5-input, for NAND, NOR, NOT, AND, OR, and XOR gate



Test Bench for 3 input, 2 output circuit debug(Before)



Test Bench for 3 input, 2 output circuit Debug(After)

## Conclusion:

This portion of the experiment was a success. The results for part 2 and part 3 were as expected after coding in the test bench. A solution for part 4 has been found and has worked properly since. The use of test benches in this software has helped further understanding of test benches and how they're implemented.